# Auditoria técnica completa — Kariri.app

**Data da auditoria:** 11 de novembro de 2026
**Escopo:** frontend, backend, banco de dados, integração, segurança e infraestrutura para lançamento do MVP
**Modo:** somente leitura e análise estática
**Especificação principal:** `prompt_auditoria_kariri_mvp.md`

## A. Diagnóstico executivo

O Kariri.app possui uma base aproveitável para um MVP, mas **não está tecnicamente pronto para lançamento operacional seguro**. O frontend tem uma SPA React/Vite com rotas, componentes, mapas, PWA, serviços HTTP, autenticação, contribuições e painel administrativo. O ZIP também contém um backend Laravel relativamente completo, com rotas públicas, autenticação Sanctum, migrations, recursos JSON, validações, contribuições, claims, favoritos, avaliações, uploads e administração.

A preparação, porém, está incompleta porque há riscos críticos de exposição de artefatos, incerteza sobre o document root da Hostinger, CORS com configuração inválida/abrangente, token bearer em `localStorage`, avaliações publicadas sem moderação, divergências entre documentação e código, status de publicação inconsistentes, uploads sem ciclo de vida completo, fallback para mocks e ausência de validação real contra MariaDB/Hostinger.

Os bloqueadores mais importantes para o lançamento são:

1. **Pacote e infraestrutura não estão seguros por construção.** O ZIP contém `.env`, log Laravel, banco SQLite, backups de PHP, SQL de deploy, `vendor`, `bootstrap`, `storage` e uploads. Isso é especialmente grave se o domínio da API continuar apontando para a raiz de `public_html`, em vez de somente para o diretório público do Laravel.
2. **CORS está quebrado ou, no mínimo, não confiável.** `config/cors.php` contém padrões regex sem delimitadores e registra no log uma falha de `preg_match`; também combina `supports_credentials=true` com paths, métodos, headers e subdomínios amplos.
3. **Reviews violam o requisito central de moderação.** `ReviewController` grava novos comentários como `published`, então conteúdo enviado pelo usuário é publicado imediatamente.
4. **A autenticação do frontend deixa o token em `localStorage`.** Um XSS no mesmo origin pode exfiltrar um bearer token válido por até 30 dias, conforme a configuração do Sanctum.
5. **Os contratos não estão congelados.** Há diferenças entre o frontend, o backend do ZIP, os artefatos Laravel no GitHub e o OpenAPI. Paginação, status, busca, cidade/região,

reviews, eventos, uploads e usuários precisam de um contrato único.

6. **Não há comprovação de execução.** Nenhum build, lint, teste, servidor, chamada HTTP, browser/E2E, migration, scanner de dependências ou teste de MariaDB foi executado nesta auditoria.

A recomendação é **não publicar ainda**. A sequência mais segura e econômica é: contenção de exposição e credenciais; definição do document root; correção de CORS e estratégia de sessão; implementação de moderação; unificação dos contratos; testes em cópia/ambiente controlado; só então uma publicação gradual e reversível.

## Verificação de materiais e acessibilidade

Os materiais foram verificados da seguinte forma:

- O ZIP enviado `/home/ubuntu/upload/_public_html(1).zip` foi aberto e extraído somente para uma área temporária de auditoria.
- O arquivo `prompt_auditoria_kariri_mvp.md` foi localizado em `prompt_auditoria_kariri_mvp.md` na raiz do ZIP e lido integralmente, incluindo as regras obrigatórias e o formato exigido para o diagnóstico.
- O frontend do GitHub `https://github.com/diegocvmoreira/kaririapp` foi acessado com sucesso. A branch padrão é `main`; a página pública informa 38 commits e contém `src`, `public`, `tests`, `docs`, manifestos e documentação.
- O repositório GitHub foi clonado em `/tmp/kariri-repo` somente para leitura. O clone estava sem alterações locais.
- O backend Laravel **não foi fornecido como repositório GitHub separado no material recebido**. O backend foi encontrado dentro do ZIP, em `kariri-api-core/`, com código executável, `artisan`, `composer.json`, `app`, `routes`, `config`, `database`, `tests` e `vendor`.
- O frontend também contém artefatos ZIP em `public/assets/aistudio/*.zip`. Eles não foram tratados como fonte canônica nem executados.
- O ZIP contém um `.env` real do backend, mas seus valores não foram reproduzidos neste relatório. A presença, o tipo e o caminho do arquivo são suficientes para caracterizar risco de empacotamento.
- Não foi possível confirmar por esta auditoria se a versão atualmente instalada na Hostinger é idêntica ao backend do ZIP, se o document root real está isolado em `public/`, nem se a API de produção está saudável. Esses itens exigem acesso autorizado ao ambiente ou testes controlados, que não foram realizados.

# B. Inventário técnico

## Frontend

O frontend está em `/tmp/kariri-repo` e usa React 19, Vite 8, TypeScript 7, React Router 7, Tailwind CSS 4, Leaflet 1.9.4, `lucide-react`, `motion` e `browser-image-compression`, conforme `package.json:16-37`.

As rotas estão centralizadas em `src/routes/AppRoutes.tsx:34-182` e cobrem:

- página inicial e exploração;
- mapa e detalhes de locais/eventos/categorias;
- favoritos, perfil e contribuições;
- cadastro, login, verificação de e-mail, recuperação e redefinição de senha;
- reivindicação de empresas;
- painel administrativo;
- páginas legais e 404.

A integração HTTP está centralizada em `src/services/api/config.ts:60-166`. A URL padrão é `https://api.kariri.app.br/api`, alterável por `VITE_API_BASE_URL`. Mocks são habilitados somente quando `VITE_USE_MOCK_DATA === 'true'`, mas vários serviços ainda possuem fallback e dados de demonstração.

O frontend possui um service worker em `public/sw.js:1-196`, manifest PWA em `public/manifest.json:1-39`, sitemap e `robots.txt`. O mapa usa Leaflet/OpenStreetMap e há mapeadores de resposta em `src/services/api/mappers/`.

Os testes do frontend são scripts TypeScript em `tests/contractValidation.ts` e `tests/placesValidation.ts`. Eles validam schemas, mocks, mapeadores e dados locais; não são testes HTTP ou E2E contra a API real.

## Backend

O backend está em `/tmp/kariri-audit/kariri-api-core` e apresenta:

- Laravel `^13.17` e PHP `^8.3`, conforme `composer.json:8-13`;
- Laravel Sanctum `^4.3`;
- Intervention Image sem faixa de versão fixada (`intervention/image-laravel: *`);
- modelos para `User`, `City`, `Category`, `Place`, `Event`, `Favorite`, `Review` e `BusinessClaim`;
- controllers públicos, autenticados e administrativos;
- Form Requests para lugares, eventos, contribuições e reviews;
- resources JSON para lugares e eventos;
- middleware `verified`, `auth:sanctum`, `admin` e Turnstile;

- migrations para usuários, cidades, categorias, lugares, eventos, favoritos, reviews, claims e tokens Sanctum;
- testes de feature concentrados em lugares, filtros, proximidade e importação;
- configuração de storage público em `config/filesystems.php:41-48,76-78`.

As rotas principais estão em `routes/api.php:42-130`. O código possui uma base funcional, mas o material tem cópias das migrations e patches em `app/database/`, `bootstrap/database/`, `deploy/` e `bootstrap/deploy/`, o que aumenta o risco de operação incorreta.

### Banco e persistência

O modelo relacional previsto inclui cidades, categorias, lugares, eventos, usuários, favoritos, reviews, claims, cache, jobs, sessões e tokens pessoais. As migrations utilizam foreign keys, JSON, enum e índices.

O arquivo `database/database.sqlite` existe no ZIP, mas a inspeção desta auditoria não o tratou como banco de produção. A presença física do arquivo em um pacote de aplicação é um risco de exposição e não prova sincronização com o MariaDB da Hostinger.

A configuração prevê MariaDB/MySQL em `config/database.php`, mas o comportamento real do schema, permissões, índices, extensões PHP e compatibilidade com MariaDB 10.6 não foi executado.

### Infraestrutura e deploy

O README do backend recomenda que o document root aponte para `kariri-api/public` (`kariri-api-core/README.md:63-69`). Entretanto, a especificação da auditoria informa que a estrutura atual está dentro de `public_html`. Essa diferença precisa ser resolvida antes de qualquer publicação.

O script `docs/laravel/deploy/deploy_pipeline.sh` do frontend/documentação presume comandos de manutenção, migrations, cache e um teste em `127.0.0.1:8000`. Isso não é uma garantia de compatibilidade com Hostinger Premium compartilhada e não deve ser usado como deploy automático sem revisão.

## C. Matriz de funcionalidades do MVP

| Funcionalidade | Situação | Evidência e conclusão |
|---|---|---|
| Explorar locais publicados | Parcialmente implementado | `app/Http/Controllers/PlaceController.php:16-116` filtra `published`, pagina, filtra cidade/categoria/rating, busca, distância e `open_now`. Não |

| | | |
|---|---|---|
| | | verificado em execução com frontend e banco real. |
| Consultar detalhes e endereço | Parcialmente implementado | `PlaceController.php:129-137` busca por slug e retorna `PlaceResource`. URL, imagens e campos precisam de smoke test. |
| Buscar locais e eventos | Parcialmente implementado | Há `/search` em `routes/api.php:70` e `SearchController.php:14-121`. O frontend também envia busca diretamente em `/places`; os dois caminhos não estão documentados como contrato único. |
| Filtrar por cidade e categoria | Parcialmente implementado | Há filtros em `PlaceController.php`, `EventController.php` e `SearchController.php`. `GET /cities?region=cariri` é usado pelo frontend, mas `CityController` não evidencia tratamento explícito do parâmetro `region`. |
| Ver locais no mapa | Parcialmente implementado | Leaflet e Haversine existem. `tests/placesValidation.ts:83-149` documenta 29 locais com coordenadas e 4 sem coordenadas. Não houve validação visual, de GPS ou de runtime. |
| Consultar eventos | Parcialmente implementado | Rotas públicas `routes/api.php:67-68`, `EventController.php:12-47` e frontend `src/services/api/events.ts`. Faltam testes de contrato, paginação e campos opcionais. |
| Criar conta | Parcialmente implementado | `AuthController.php:18-65` valida senha forte e envia verificação de e-mail. A rota ativa não usa Turnstile. Não houve teste de SMTP ou fluxo de confirmação. |

| | | |
|---|---|---|
| Login e sessão | Parcialmente implementado | `AuthController.php:68-96` emite token Sanctum; frontend guarda o token em `localStorage` ( `src/services/api/auth.ts` e `config.ts` ). Não houve teste HTTP real. |
| Perfil e recuperação de senha | Parcialmente implementado | Existem endpoints e páginas correspondentes. `/user` retorna model bruto, enquanto `/me` usa payload explícito. SMTP, expiração e resposta real não foram verificados. |
| Favoritos | Parcialmente implementado | Rotas `routes/api.php:98-100` e serviço frontend existem. Faltam testes de ownership, duplicidade, deleção e contrato. |
| Contribuir com local | Implementado no fluxo básico | `ContributionController.php:21-37` força `pending` ; request valida cidade, categoria e dados. Moderação administrativa e persistência real não foram testadas. |
| Contribuir com evento | Implementado no fluxo básico, com perda de campos | `ContributionController.php:39-55` normaliza aliases e remove `cover_image` , `place_name` , `is_free` e `organizer` . O frontend coleta mais campos que o backend persiste. |
| Avaliar e comentar locais | Incompatível com regra de moderação | `ReviewController.php:39-46` cria review como `published` . O MVP exige que conteúdo de usuário passe por moderação. |
| Reivindicar estabelecimento | Parcialmente implementado | Usuário pode enviar claim e admin pode revisar. `proof` é apenas string, não upload de prova; não há verificação explícita de local publicado ou claim aprovado concorrente. |
| Moderação administrativa | Parcialmente implementado | Grupo admin em `routes/api.php:118-130` e middleware |

| | | |
|---|---|---|
| | | `EnsureUserIsAdmin.php:11-15`. Há CRUD e claims, mas não existe fluxo dedicado para moderar reviews e a cobertura é pequena. |
| Upload de imagens | Parcialmente implementado | `ImageUploadController.php:16-57` valida e reencoda WebP. Não há quota, limpeza de órfãos, associação obrigatória a entidade, antivírus ou teste. |
| PWA | Implementado estaticamente | Manifest e service worker existem. Instalação, atualização e fallback offline não foram executados. |
| Responsividade e acessibilidade | Parcialmente implementado | Há navegação mobile, estados e vários `aria-label`, mas existem labels sem associação `htmlFor` / `id`, por exemplo em `src/pages/CreatePlacePage.tsx:383-428`. Não houve inspeção visual ou leitor de tela. |

# D. Relatório de segurança

## P0 — riscos que bloqueiam a publicação

## S1. Exposição potencial de `.env`, logs, banco e código interno — crítica/alta

**Evidência:**

- `/tmp/kariri-audit/kariri-api-core/.env`;
- `/tmp/kariri-audit/kariri-api-core/storage/logs/laravel.log`;
- `/tmp/kariri-audit/kariri-api-core/database/database.sqlite`;
- diretórios `app`, `bootstrap`, `config`, `routes`, `vendor` e `storage`;
- backups `*.backup.php` em `app/`;
- SQL de deploy em `deploy/`, `app/deploy/` e `bootstrap/deploy/`;
- uploads reais em `public/storage/uploads/users/6/`;

- ZIPs versionados em `/tmp/kariri-repo/public/assets/aistudio/`.

O `.gitignore` não reduz o risco de um arquivo já incluído no pacote ou copiado para `public_html`. Se o domínio apontar para a raiz Laravel, e não somente para `public/`, arquivos de configuração, logs, banco e código interno podem ser baixados.

**Consequência:** vazamento de credenciais, PII, stack traces, estrutura de banco e código; possibilidade de comprometimento da aplicação caso qualquer credencial tenha sido real.

**Recomendação:** produzir release por allowlist; remover `.env`, logs, SQLite, backups, SQL e ZIPs; revisar histórico e rotacionar credenciais potencialmente expostas; manter a raiz Laravel fora do document root; bloquear acesso a artefatos internos no servidor.

## S2. CORS inválido e excessivamente amplo — alta

**Evidência:** `config/cors.php:8-12` inclui `'*'` em `paths`; `:14` permite todos os métodos; `:31` todos os headers; `:41` define `supports_credentials=true`; `:26-29` contém padrões regex sem delimitadores válidos e permite subdomínios amplos.

O arquivo `storage/logs/laravel.log` contém evidência histórica de `preg_match(): No ending delimiter '^' found` no componente de CORS. Isso é evidência registrada no material, não um novo teste realizado nesta auditoria.

**Consequência:** preflight pode falhar, a integração pode ficar intermitente e a superfície de confiança de origens fica maior que o necessário.

**Recomendação:** decidir se a aplicação usará Bearer-only ou cookies stateful; restringir `paths` aos necessários; listar origens exatas de produção e desenvolvimento autorizado; corrigir os delimitadores regex; remover wildcard e subdomínios não necessários; confirmar a configuração efetiva depois de uma validação controlada.

## S3. Token bearer em `localStorage` — alta

**Evidência:** `src/services/api/config.ts:65-77` lê `kariri_auth_token` e monta o header Authorization. `src/services/api/auth.ts` grava o token e o perfil. `config/sanctum.php:53` define expiração de 30 dias.

**Consequência:** qualquer XSS ou script de terceiro executado no origin pode copiar o token e usá-lo até expirar ou ser revogado.

**Recomendação:** preferir cookie `HttpOnly`, `Secure`, `SameSite` com CSRF adequado para uma SPA same-site. Se Bearer for mantido, reduzir TTL, definir rotação/revogação, reforçar CSP, minimizar terceiros e documentar formalmente a decisão.

## S4. Reviews publicadas sem moderação — alta

**Evidência:** app/Http/Controllers/Api/ReviewController.php:39-46 grava status => 'published', enquanto a listagem pública em :17-22 retorna reviews publicadas. O prompt exige moderação para conteúdo enviado por usuários.

**Consequência:** comentário ofensivo, spam ou conteúdo malicioso é publicado imediatamente; rating e contagem também são atualizados.

**Recomendação:** gravar como pending; criar aprovação/rejeição administrativa; atualizar rating e contagem somente com itens publicados; aplicar escaping normal na saída, antispam e testes de transição.

## S5. Script de deploy com fallback previsível e operações destrutivas — alta

**Evidência:** docs/laravel/deploy/deploy_pipeline.sh contém fallback literal para segredo de bypass, roda comandos de manutenção/migration/cache e faz health check em localhost. O README oferece ainda estratégias alternativas de migration e SQL manual.

**Consequência:** bypass previsível, migration parcial, interrupção em hospedagem compartilhada e falso positivo de saúde.

**Recomendação:** falhar se segredo não for fornecido; nunca usar valor padrão; separar deploy de schema; não colocar a aplicação em manutenção automaticamente; usar health check da URL real; preparar rollback e janela controlada.

# P1 — riscos importantes antes do MVP

## S6. Turnstile não está aplicado às rotas efetivas de autenticação — média/alta

As versões com Turnstile estão comentadas em routes/api.php:72-74. As rotas efetivas de register, login e forgot-password em :75-84 usam apenas throttle. O middleware VerifyTurnstile existe e funciona por código, mas não protege os fluxos mais sujeitos a abuso.

Recomenda-se aplicar proteção antiabuso compatível a cadastro, login e recuperação, combinando limites por IP, e-mail/conta e tratamento de falha.

## S7. Upload público sem governança de ciclo de vida — média/alta

ImageUploadController.php:18-25 valida arquivo, imagem, mime e tamanho de 5 MB, mas :44-56 salva em disco público, retorna URL pública e não associa o arquivo a uma entidade. Não há quota, limpeza, substituição, retenção, antivírus ou teste de imagem malformada. O frontend também aceita URLs externas em contribuições.

Recomenda-se limitar quota, dimensões/pixels e frequência, permitir apenas storage controlado, limpar órfãos, impedir execução de conteúdo, definir headers e remover ou

restringir URLs externas.

## S8. Endpoint `/api/user` retorna o model bruto — média

`routes/api.php:49-51` retorna `$request->user()` diretamente, enquanto `AuthController::userPayload()` em `:239-262` define um contrato explícito. O model oculta senha e remember token, mas o endpoint pode expor atributos internos atuais ou futuros e diverge do `/me`.

Recomenda-se remover a duplicidade ou retornar um resource/payload explícito e versionado.

## S9. Autorização depende de rota, sem defesa em profundidade — média

`StorePlaceRequest.php:12-15` e `StoreEventRequest.php:10-13` retornam `authorize=true`. No fluxo atual, as rotas administrativas são protegidas pelo grupo `auth:sanctum`, `verified`, `admin`, mas uma reutilização futura do request em rota indevida poderia abrir acesso.

Recomenda-se Policies/authorization específica por operação, ownership e testes sistemáticos 401/403 por papel.

## S10. Status de publicação inconsistentes — média/alta

`PlaceController.php` público exige `published`; `SearchController.php:27-36` aceita `active` e `published`; o SQL de deploy converte `active` em `published`; `StoreEventRequest.php:105-114` aceita também `active`. Isso pode fazer itens aparecerem na busca e desaparecerem no catálogo ou mudar de visibilidade após patch.

Recomenda-se definir uma máquina de estados única, migrar/validar dados antes da mudança e cobrir todos os status com testes.

## S11. Dependência PHP sem versão fixada — média

`composer.json:10` usa `intervention/image-laravel: *`. Uma instalação futura pode selecionar uma versão incompatível com PHP, Laravel ou GD da Hostinger.

Recomenda-se fixar uma faixa compatível, revisar `composer.lock` e incluir `composer audit` no CI.

## S12. Logs e informações operacionais dentro do pacote — média

O log tem aproximadamente 1,3 MB e registra erros, caminhos, stack traces e dados de infraestrutura. Mesmo que não contenha uma credencial utilizável, não deve ser distribuído com o release.

Recomenda-se rotação, nível adequado em produção, `APP_DEBUG=false`, retenção controlada e exclusão do pacote público.

### Itens não confirmados por não execução

Não foi executado fuzzing de SQL injection/XSS, scanner de dependências, browser, HTTP, scanner de arquivos públicos, análise de CSP efetiva ou teste de permissões. A ausência de evidência nesses testes não deve ser interpretada como prova de segurança.

## E. Matriz de integração frontend + backend

| Área | Frontend | Backend/artefatos | Classificação |
|---|---|---|---|
| Listagem de locais | `src/services/api/places.ts:31-91` usa cidade, categoria, rating, aberto, busca, ordenação e coordenadas | `PlaceController.php:16-116` implementa grande parte dos filtros e paginação | Parcial; não verificado em execução |
| Detalhe de local | `places.ts:166-187` usa `/places/{slug}` | `routes/api.php:62-64` e `PlaceController.php:129-137` usam slug | Compatível nominalmente; runtime não verificado |
| Busca textual | frontend envia `q` para `/places` e `/search` | ZIP implementa `q` no `PlaceController`; `SearchController` também existe | Parcial; documentação GitHub e caminhos podem divergir |
| Eventos | `events.ts:63-147` lista e detalha | `routes/api.php:67-68` e `EventController.php:12-47` existem | Parcial; campos e paginação não foram validados |
| Cidades | `cities.ts` usa `/cities`, `/cities/{slug}` e `?region=cariri` | `CityController` implementa lista/busca/detalhe, mas não evidencia filtro `region` | Parcial/incompatível potencial |
| Categorias | `categories.ts` usa lista/detalhe e fallback em 404 | `routes/api.php:59-60` existem | Parcial; contrato de erro/fallback mascara problemas |
| Autenticação | `/login`, `/register`, `/me`, `/logout`, reset, Bearer em `localStorage` | `AuthController`, Sanctum e verificação de e-mail existem | Parcial; estratégia de sessão/CORS não está coerente nem testada |

| | | | |
|---|---|---|---|
| `/me` e `/user` | frontend possui fallback entre os dois | `/me` usa payload explícito; `/user` devolve model bruto | Inconsistente |
| Favoritos | serviço chama GET/POST/DELETE `/favorites` | rotas autenticadas existem | Parcial; ownership e contrato não testados |
| Reviews | frontend usa `POST /places/{id}/reviews` | ZIP usa `POST /places/{place}/reviews`; binding implícito tende a ID | Parcial; precisa congelar ID/slug e corrigir moderação |
| Contribuição de local | envia `/contributions/places` com imagens/URLs | backend força `pending` e valida request | Parcial; upload e campos precisam contrato formal |
| Contribuição de evento | coleta `is_free`, `organizer`, `cover_image` e aliases | backend remove alguns desses campos antes de persistir | Incompatível/parcial |
| Upload | `/uploads/images` com FormData | controller valida e converte para WebP | Parcial; storage, quota, associação e segurança não fechados |
| Claims | user/admin usam `/business-claims` | controllers user/admin existem | Parcial; prova é string e não há testes de ownership/concorrência |
| Admin | guarda frontend verifica `role=admin` | grupo backend usa `auth:sanctum`, `verified`, `admin` | Parcial; guarda frontend é apenas UX e não substitui backend |
| Paginação | frontend aceita `data`, mas frequentemente reduz resposta a array e descarta `meta` | Laravel retorna recursos paginados com `data`, `links`, `meta` | Divergência potencial |
| OpenAPI | `docs/schemas/openapi.json` documenta somente subconjunto | código possui mais rotas do que o OpenAPI | Incompleto |

## Observação sobre os repositórios

O GitHub acessível contém documentação e artefatos Laravel em `docs/laravel`, mas não contém um backend Laravel completo na raiz. O ZIP contém outro backend, com código executável. Portanto, não se deve assumir que `docs/laravel` e `kariri-api-core/` representam exatamente o mesmo commit, schema ou deployment.

## F. Lista consolidada de problemas

| ID | Problema | Caminhos | Gravidade | Consequência | Solução recomendada |
|---|---|---|---|---|---|
| P0-01 | Pacote contém `.env`, logs, SQLite, backups, SQL, vendor e caches | ZIP; `kariri-api-core/.env`, `storage/logs`, `database/database.sqlite`, `app/**/*.backup.php` | Crítica | Vazamento de segredo, PII e código | Release por allowlist; remover artefatos; revisar histórico e rotacionar segredos |
| P0-02 | Document root pode apontar para raiz de `public_html` | `kariri-api-core/README.md:63-69`; contexto Hostinger | Crítica | Acesso público a internos do Laravel | Apontar domínio somente para `public/`; manter raiz fora da web |
| P0-03 | CORS inválido/amplo com credenciais | `config/cors.php:8-41`; `storage/logs/laravel.log` | Alta | Preflight quebrado e origens excessivas | Fixar origens, regex, paths e estratégia de sessão |
| P0-04 | Reviews publicadas diretamente | `app/Http/Controllers/Api/ReviewController.php:39-46` | Alta | Conteúdo de usuário publicado sem moderação | Status `pending`, fluxo admin e rating só para publicados |
| P0-05 | Bearer token em `localStorage` | `src/services/api/config.ts:65-77`; `auth.ts` | Alta | Roubo de sessão em XSS | Cookie HttpOnly ou mitigação formal de Bearer |
| P0-06 | Deploy possui fallback de segredo e | `docs/laravel/deploy/deploy_pipeline.sh` | Alta | Bypass e indisponibilidade/migration indevida | Remover fallback; separar |

| | | | | | |
|---|---|---|---|---|---|
| | migration automática | | | | schema/deploy; rollback |
| P1-01 | Turnstile não aplicado às rotas ativas de auth | `routes/api.php:72-86` ; `src/services/api/auth.ts` | Média/alta | Abuso de login, cadastro e reset | Aplicar proteção antiabuso e limites por conta/IP |
| P1-02 | Upload sem quota, associação e limpeza | `ImageUploadController.php:16-57` | Média/alta | Abuso de storage e arquivos órfãos | Quotas, associação, retenção, validação de pixels e storage seguro |
| P1-03 | Status `active` / `published` / `archived` divergentes | `PlaceController.php` , `SearchController.php` , requests, SQL | Média/alta | Visibilidade inconsistente | Máquina de estados única e migração validada |
| P1-04 | Contrato de evento perde campos | `events.ts:249-285` ; `ContributionController.php:39-55` | Média | Dados coletados não aparecem depois | Definir campos MVP e schema único |
| P1-05 | `/api/user` retorna model bruto | `routes/api.php:49-51` | Média | Exposição e contrato instável | Resource/payload explícito |
| P1-06 | Paginação/envelope não padronizados | controllers/resources; serviços frontend | Média | Contagem e paginação incorretas | Normalizadores e fixtures de contrato |
| P1-07 | Mocks/fallback podem esconder falhas reais | `src/mocks/*` ; `cities.ts` ; serviços API | Média | Demo parece funcional mesmo com API falha | Mock apenas desenvolvimento; falha explícita em produção |
| P2-01 | Dependência `intervention/image-laravel:*` | `composer.json:10` | Média | Instalação não reprodutível | Fixar versão e auditar lock |

| | | | | | |
|---|---|---|---|---|---|
| P2-02 | CI cobre somente frontend | `.github/workflows/ci.yml:1-22` | Média | Regressões no backend passam despercebidas | CI PHP/Composer/PHPUnit/SCA |
| P2-03 | Migrations e patches duplicados | `database/`, `app/database/`, `bootstrap/database/`, `deploy/` | Média | Operação e schema divergentes | Uma fonte canônica e estratégia única |
| P2-04 | Labels/foco/acessibilidade incompletos | `src/pages/CreatePlacePage.tsx:383-428` | Baixa/média | Formulários menos acessíveis | `id` / `htmlFor`, `aria-live`, foco em modal/erro |
| P2-05 | Ausência de testes para fluxos críticos | `tests/`, backend `tests/Feature` | Alta para release | Falha não detectada em auth, upload, claims e admin | Testes HTTP, integração, E2E e permissões |

# G. Plano priorizado de implementação do MVP

## Etapa 1 — Segurança e contenção crítica

**Esforço:** médio.
**Dependências:** nenhuma para o diagnóstico; acesso autorizado à Hostinger para execução posterior.
**Riscos:** rotação de credenciais pode invalidar sessões; qualquer alteração em produção deve ter backup e janela controlada.

Ações planejadas:

- produzir um artefato de release somente com arquivos necessários;
- remover `.env`, logs, SQLite, backups, SQL, ZIPs e caches;
- revisar histórico e rotacionar credenciais que tenham sido reais;
- configurar `APP_DEBUG=false`;
- apontar o domínio somente para `public/`;
- restringir CORS e corrigir regex;
- escolher cookie HttpOnly ou Bearer com política formal;

- remover fallback de segredo do deploy.

**Critérios objetivos:** nenhum segredo/backup/log/SQLite no pacote; nenhum arquivo interno acessível por URL; preflight autorizado para o frontend real; respostas sem stack trace; estratégia de autenticação aprovada.

## Etapa 2 — Estabilização do backend e do schema

**Esforço:** médio/alto.
**Dependências:** backup e confirmação do schema atual.
**Riscos:** migration incompatível, enum divergente e falta de extensão GD/CLI na hospedagem.

Ações planejadas:

- escolher uma única estratégia: Artisan migrations ou SQL manual controlado;
- remover cópias operacionais de migrations/patches da distribuição;
- conferir tabela `migrations`, foreign keys, enum/status, índices e campos efetivos;
- fixar `intervention/image-laravel`;
- substituir `/user` por payload explícito;
- implementar Policies/ownership e papéis necessários;
- confirmar storage link, permissões, GD e SMTP.

**Critérios objetivos:** schema reproduzível em cópia MariaDB 10.6; migrations idempotentes e revisadas; autenticação, autorização, storage e e-mail comprovados em ambiente controlado.

## Etapa 3 — Contrato frontend/API

**Esforço:** médio/alto.
**Dependências:** etapa 2 e decisão de produto sobre reviews, status e campos de eventos.
**Riscos:** alterar nomes de campos pode exigir atualização de vários serviços e componentes.

Ações planejadas:

- definir OpenAPI como fonte única;
- padronizar envelopes `{data, meta, links}`;
- decidir se review usa ID ou slug;
- alinhar `/places`, `/search`, `/events`, cidades/região e paginação;
- alinhar status `draft/pending/published/rejected/archived`;
- alinhar campos de evento, uploads, claims e usuário;

- remover fallbacks que silenciam 404/500 em produção.

**Critérios objetivos:** cada chamada do frontend tem endpoint, método, payload, resposta e códigos de erro documentados; fixtures de contrato passam para sucesso, validação, autenticação e autorização.

## Etapa 4 — Funcionalidades essenciais do MVP

**Esforço:** alto.
**Dependências:** etapas 1–3.
**Riscos:** publicação indevida de conteúdo e inconsistência de rating/status.

Ações planejadas:

- catálogo publicado de locais, com cidade, categoria, endereço e mapa;
- agenda de eventos com estados e datas;
- cadastro de contas e verificação de e-mail;
- contribuições sempre pendentes;
- reviews pendentes, com aprovação/rejeição administrativa;
- claims com fluxo mínimo de análise;
- favoritos com ownership e deleção segura;
- painel admin para locais, eventos, cidades, usuários, claims e reviews;
- upload controlado, associado à entidade correta.

**Critérios objetivos:** matriz de aceitação por papel `user`, `business` e `admin`; nenhum conteúdo enviado por usuário aparece antes da aprovação; conteúdo rejeitado não é público.

## Etapa 5 — Interface, responsividade e acessibilidade

**Esforço:** médio.
**Dependências:** contratos estáveis.
**Riscos:** regressões visuais e de navegação em dispositivos pequenos.

Ações planejadas:

- revisar 360 px, tablet e desktop;
- associar labels e campos;
- garantir foco de modal e mensagens `aria-live`;
- revisar loading, vazio, offline e erro;
- validar imagens, contraste, teclado e navegação mobile;

- confirmar instalação e atualização do PWA.

**Critérios objetivos:** revisão manual nos viewports-alvo; fluxos principais navegáveis por teclado; erros compreensíveis; nenhum caminho principal depende de mock quando `VITE_USE_MOCK_DATA=false`.

## Etapa 6 — Testes e validação

**Esforço:** alto relativo ao ambiente compartilhado.
**Dependências:** etapas 1–5.
**Riscos:** inexistência de staging independente; não usar produção para testes destrutivos.

Ações planejadas:

- CI frontend e backend;
- lint, build, PHPUnit, `composer validate`, `composer audit` e scanner de dependências;
- testes HTTP de auth, verificação, reset, roles, favorites, claims, reviews, upload e eventos;
- testes de contrato e paginação;
- smoke tests somente de leitura antes de fluxos de escrita;
- teste controlado de MariaDB/Hostinger;
- revisão PWA, acessibilidade e mobile.

**Critérios objetivos:** build reproduzível; testes críticos verdes; 401/403/409/422/429 cobertos; rollback documentado; nenhum teste contra produção sem autorização expressa.

## Etapa 7 — Preparação para publicação

**Esforço:** médio.
**Dependências:** todos os critérios anteriores.
**Riscos:** indisponibilidade, cache antigo, permissões e divergência de variáveis.

Ações planejadas:

- release allowlist-only;
- document root em `public/`;
- variáveis fora do bundle;
- backup verificável;
- procedimento reversível de upload e cache;
- migrations separadas do deploy de código;
- health check no domínio real;

- monitoramento de erros sem PII.

**Critérios objetivos:** checklist Hostinger aprovado; API pública responde somente endpoints esperados; internos não respondem; rollback testado em cópia; smoke dos fluxos do MVP concluído.

# H. Funcionalidades para versões futuras

Podem ficar fora do primeiro lançamento, desde que não prejudiquem o núcleo:

- analytics avançado e relatórios estratégicos completos;
- recomendações sofisticadas e locais semelhantes;
- busca/indexação avançada;
- geocodificação automática;
- recorrência complexa de eventos;
- fila/worker e notificações assíncronas;
- S3/CDN e otimização avançada de mídia;
- reviews editáveis, denúncias e reputação avançada;
- múltiplos níveis de moderador;
- importação em massa além do fluxo mínimo de administração;
- favoritos de eventos, caso não sejam indispensáveis ao primeiro lançamento;
- integrações de calendário e recursos sociais.

Não devem ser adiados: segurança do deploy, autenticação e autorização, catálogo, busca básica, mapa básico, eventos, contribuições moderadas, reviews moderadas, claims mínimos, painel admin protegido, upload seguro e testes de contrato.

# I. Parecer final

## O projeto está tecnicamente preparado para um MVP?

**Não.** Existe uma base reaproveitável e boa parte da experiência está implementada, mas os riscos de segurança, a configuração de infraestrutura, a falta de moderação de reviews, a divergência de contratos e a ausência de validação em ambiente compatível impedem classificar o conjunto como pronto.

## O que impede o lançamento?

Principalmente: exposição potencial de `.env` /logs/SQLite/código; document root incerto; CORS inválido ou excessivo; token em `localStorage`; reviews publicadas sem moderação; Turnstile desligado nas rotas críticas; status e envelopes inconsistentes; upload sem governança completa; CI sem backend; e ausência de testes reais contra MariaDB/Hostinger.

## Quais são os maiores riscos?

Os maiores riscos são vazamento de credenciais/PII, comprometimento de sessão em caso de XSS, abuso de auth/uploads/reviews, publicação de conteúdo impróprio e indisponibilidade causada por migration/deploy inadequado em hospedagem compartilhada.

## O que pode ser aproveitado sem reconstrução?

A estrutura React/Vite, o design system, as rotas, os serviços e mapeadores frontend, o Leaflet/OpenStreetMap, o PWA, as rotas públicas Laravel, Sanctum após decisão formal, validações de contribuições, `PlaceResource` / `EventResource`, o reprocessamento de imagens para WebP, o middleware admin e os testes/fixtures existentes podem ser aproveitados.

## Qual é a sequência mais segura e econômica?

1. Conter exposição e revisar/rotacionar credenciais.
2. Corrigir document root, CORS e estratégia de autenticação.
3. Unificar migrations/schema/status e remover duplicidades operacionais.
4. Implementar moderação de reviews e reforçar Policies, uploads e antiabuso.
5. Congelar OpenAPI e corrigir integração frontend/API.
6. Adicionar testes de contrato, backend, segurança e E2E controlados.
7. Publicar somente após checklist reversível da Hostinger e smoke tests autorizados.

Nenhuma implementação foi iniciada. A auditoria termina neste ponto e aguarda autorização expressa para qualquer correção, teste de execução que altere estado, migration, deploy ou modificação futura.

# Referências


[1] Repositório GitHub público do frontend Kariri.app
[2] Documentação técnica pública do Kariri.app
[3] Manifesto de dependências do frontend Kariri.app
[4] Especificação de produto pública do Kariri.app
[5] OpenAPI público do Kariri.app